NotifyEngine

ML-Powered Adaptive Notification Routing

Sprint 2 Presentation

Team Algo-Rhythms | CS 691 | Pace University | Spring 2026

Agenda

01 Team Roles & Responsibilities

02 Improvements from Professor Feedback

03 Project Description

04 Team Working Agreement

05 Personas

06 MVP Definition

07 Technologies & Algorithms

08 Architecture & Diagrams

09 Sprint 1 Recap

10 Product Backlog & Sprint 2 Backlog

11 Test Cases & Stories Completed

12 Metrics (Velocity, Burndown, Ratios)

13 Retrospective & Sprint 3 Plan

14 Live Demo & GitHub

TEAM ROLES

Mohammed N. Yusif

Security Engineer/ Project Manager

Krish Dhorajiya

AI Engineer / Tester

Rhythm Patel

AI Engineer / Developer

TEAM ROLES

Salvatore Ardisi

SCRUM Master / Developer

Dhawalshree Mengane

Data Analyst / Developer

Zeeya Ramani

Data Analyst / Developer

Improvements from Professor Feedback

Sprint 0 Feedback

- Give individual introductions while presenting so that each member is properly introduced in the presentation.
- The recorded video had low resolution when posted — ensure better video quality and resolution in future recordings.

Sprint 1 Feedback

- **Show Complete Data:** Avoid partial screenshots so the audience doesn't lose context.
- **Remove Clutter:** Hide default spreadsheet headers (A, B, C) to reduce visual noise.
- **Maintain Consistency:** Ensure uniform formatting and styling across all slides.

Project Description

NotifyEngine is a multi-tenant notification delivery platform with machine learning-powered adaptive channel routing.

The Problem

Developers integrate 3-5 separate notification services, write custom routing logic, and have no data on which channel actually reaches each user.

Our Solution

One API call. The system decides email, SMS, WebSocket, or webhook based on per-recipient engagement data. An XGBoost model learns which channel works best for each user.

Key Differentiator

Competitors (Courier, Knock, Novu) require manual channel priority rules. NotifyEngine learns and adapts automatically from delivery outcome data.

Team Working Agreement

Meetings

Tuesdays & Thursdays via Zoom + async Slack standups 3x/week

Deadlines

Individual work due 48 hours before sprint deadline for integration

Communication

All project discussion in Slack (not WhatsApp/text)

Accountability

Missed deadlines documented; escalation path defined

Code

All changes via pull requests — no direct pushes to main/develop

Quality

npm audit + security scan before every PR merge

Personas

CUSTOMER

Backend Engineer

Manages 4 notification SDKs with no delivery analytics

CUSTOMER

Product Manager

Cannot measure which notification channels drive engagement

CUSTOMER

Growth Lead

Manually A/B tests channels with no ML-driven optimization

END-USER

Mobile-first consumer

Receives redundant notifications across channels

END-USER

Professional user

Important alerts lost in notification overload

END-USER

Casual app user

Disengages when notifications feel irrelevant

MVP - Core Notification Flow

01 Developer sends POST /v1/notifications with API key

02 Authentication - SHA-256 hash lookup, identify tenant

03 Validation - Zod schema validates all fields

04 Create notification record (status: pending)

05 Enqueue BullMQ job, return 202 Accepted

06 Worker picks up job from Redis queue

07 Channel selection - static fallback OR adaptive (XGBoost + epsilon-greedy)

08 Delivery - Nodemailer / Socket.IO / webhook / mock SMS

09 Record delivery outcome + feature vector

10 Engagement tracking - email pixel, WebSocket ack

11 Developer checks status via GET /v1/notifications/:id

Technologies

API Server

Node.js 20 + TypeScript

Express

Pino (structured JSON)

ML Service

Python 3.11 + FastAPI

XGBoost + scikit-learn

Pydantic validation

Data Layer

PostgreSQL 16 + RLS

Redis 7 + BullMQ

Zod schemas

Frontend

React 18 + TypeScript

Vite + Tailwind

Socket.IO client

Real-time

Socket.IO (WebSocket)

CI/CD

GitHub Actions + Docker

Testing

Vitest + pytest

Monorepo

Turborepo

All services containerized with Docker. Monorepo managed by Turborepo

Algorithms

XGBoost Binary Classifier

Gradient boosted decision tree ensemble predicting P(engagement | features) per (recipient, channel) pair. 100 trees, max depth 6, learning rate 0.1, optimized on AUC-ROC.

Bootstrap model: AUC-ROC 0.7558, F1 0.6807, 8,000 training samples

Epsilon-Greedy Exploration

90% exploit: select channel with highest predicted engagement. 10% explore: select random channel to discover behavior changes. Rate adjusts by data volume.

0-500: 50% explore | 500-2K: 20% | 2K+: 10%

Feature Vector (15 dimensions)

channel_type, hour_of_day, day_of_week, is_weekend, historical_success_rate, historical_engagement_rate, hours_since_last_engagement, hours_since_last_success, avg_latency_ms, attempts_30d, notifications_sent_24h, notifications_sent_7d, notification_priority_score, content_length, channel_health

ML Training Pipeline

Cold Start

0-500 notifications

Exploration: 50%

Not enough data to train. Static fallback with heavy exploration. Every delivery generates labeled training data.

Initial Model

500-2,000 notifications

Exploration: 20%

First XGBoost model trained. AUC-ROC ~0.70. Retrains every 6 hours. Better-than-random channel selection.

Mature Model

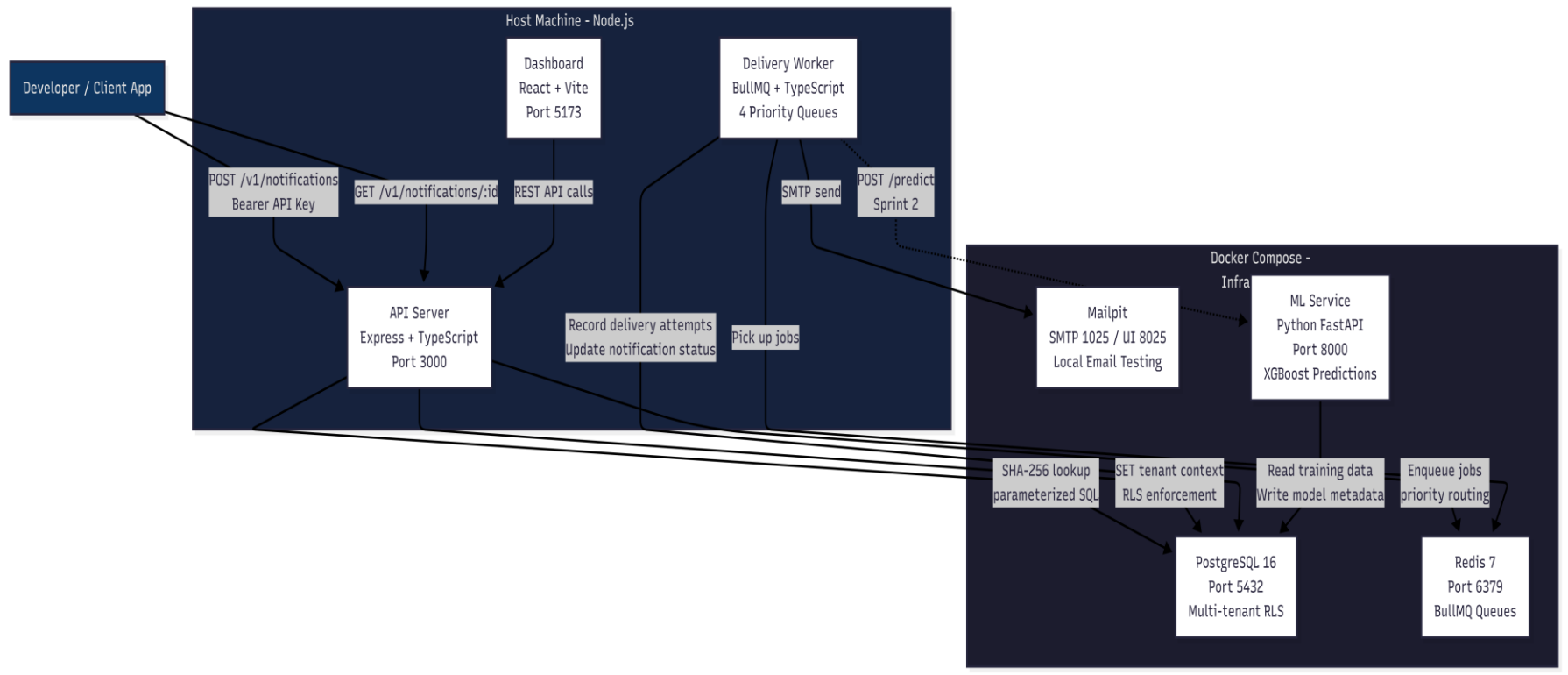
2,000+ notifications

Exploration: 10%

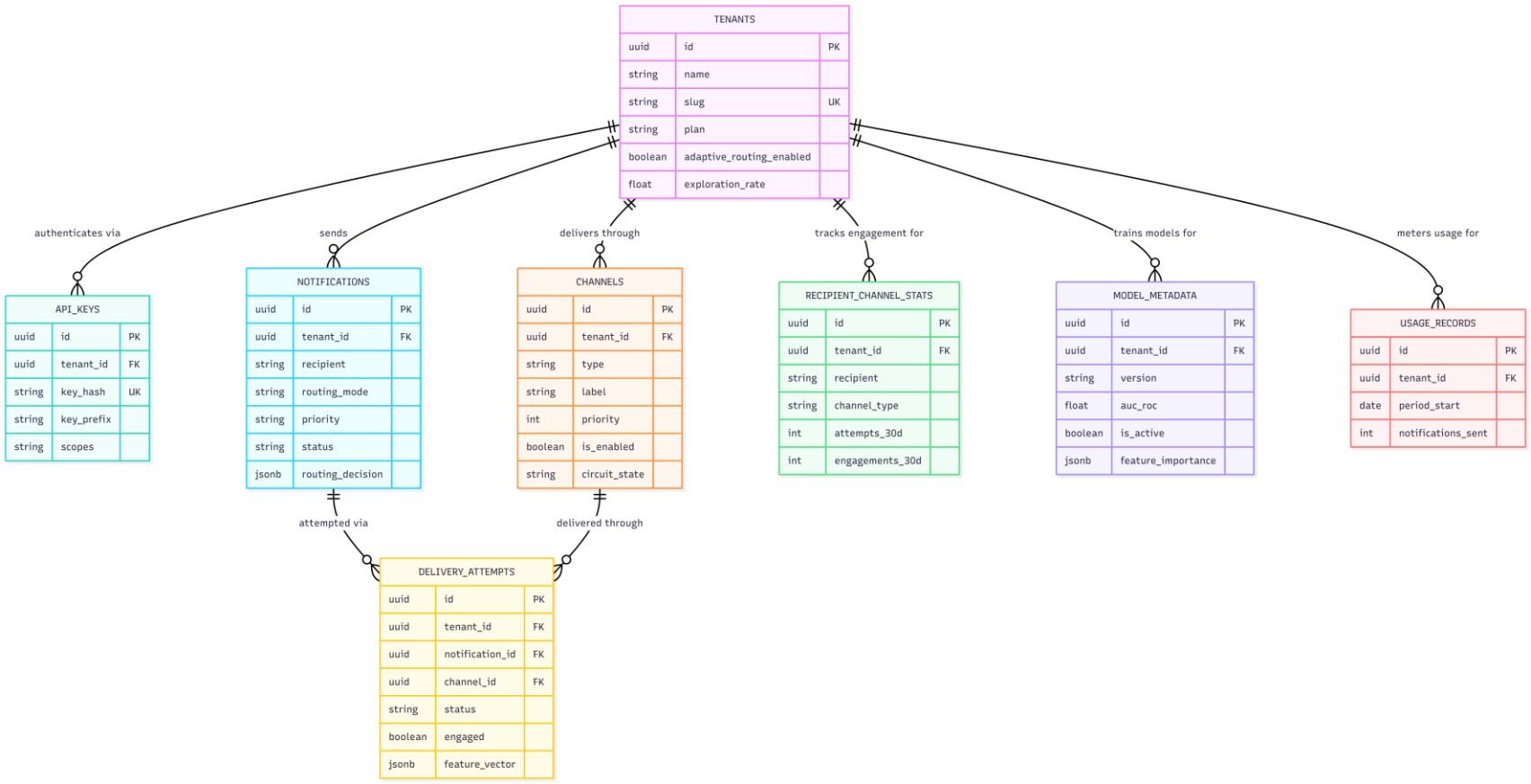
Strong per-channel predictions. AUC-ROC ~0.80+. Feature importance visible. Retrains on rolling window.

Retraining: query delivery_attempts, extract features, train new model, promote only if AUC-ROC improves over current.

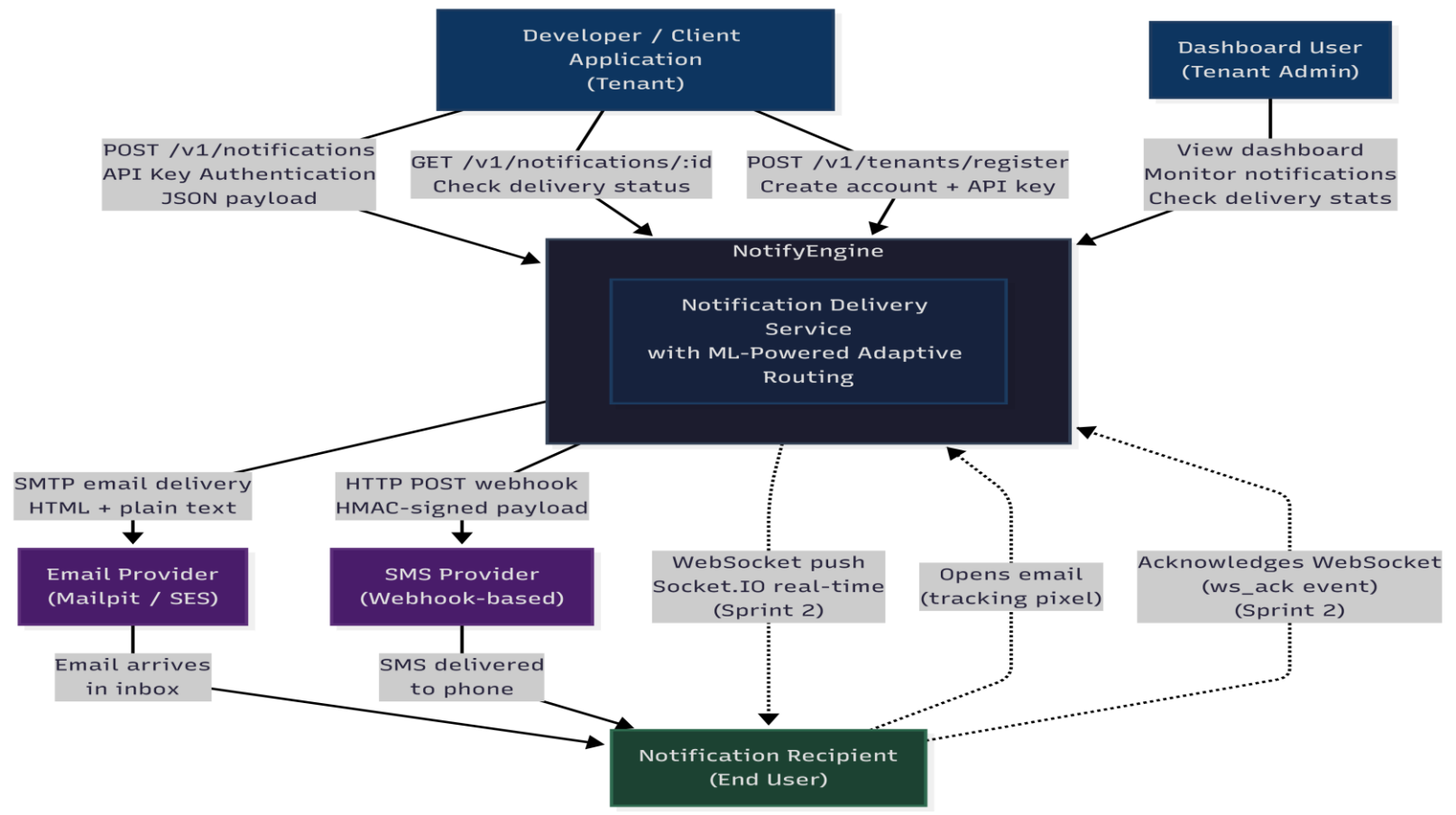
Architecture Diagram



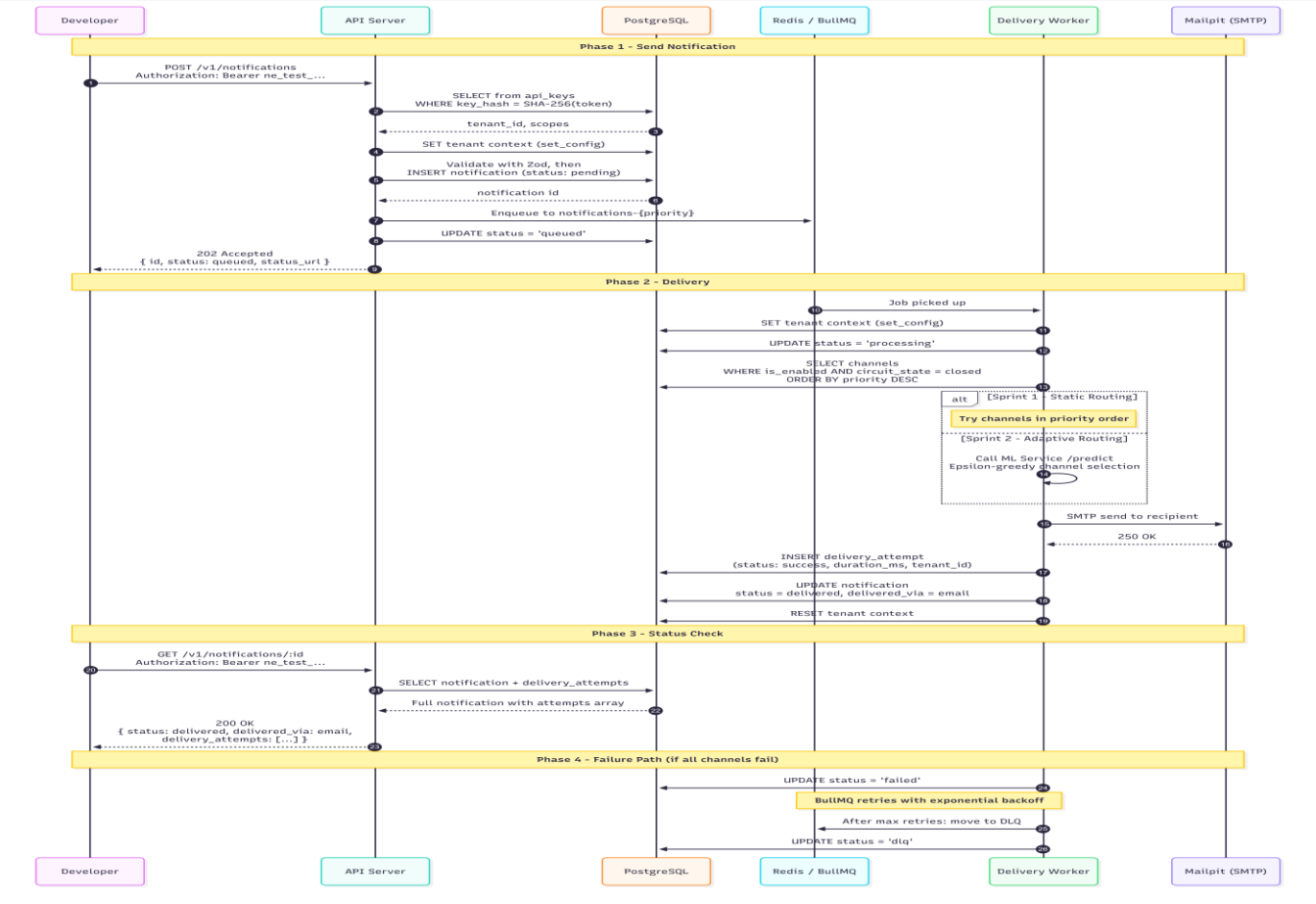
Conceptual Diagram



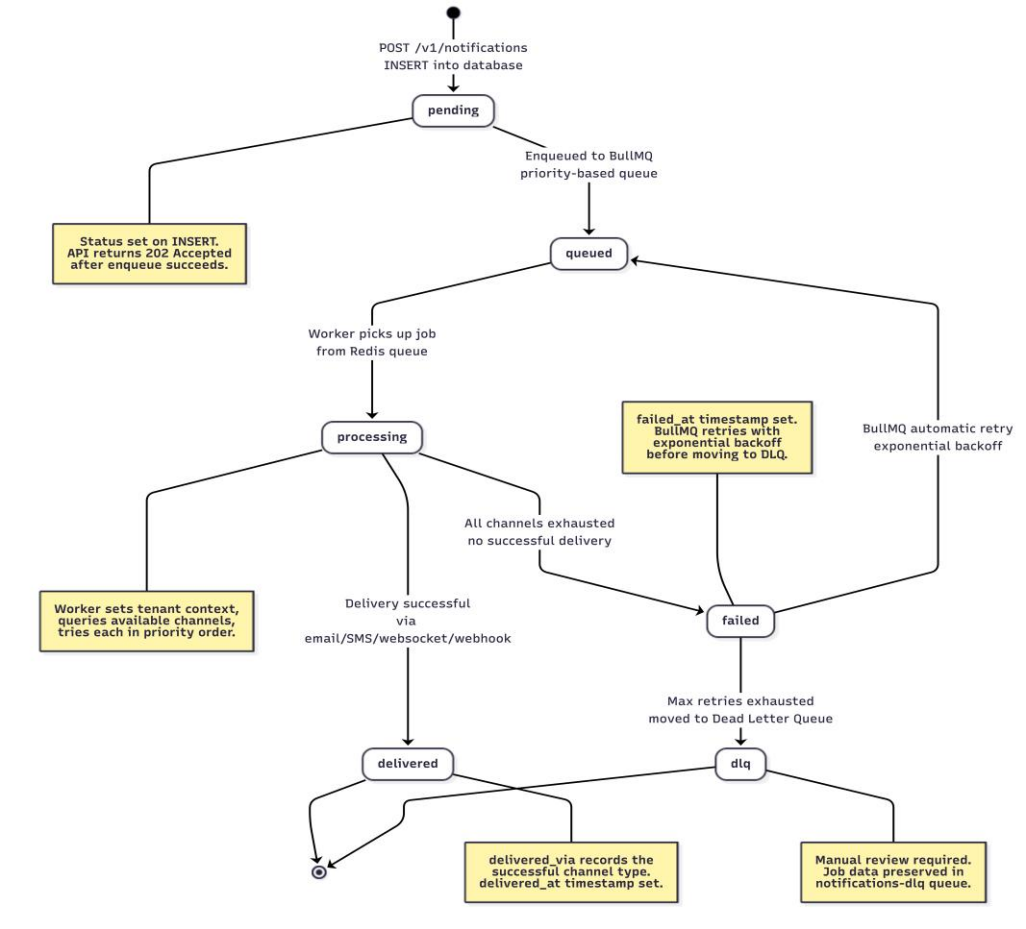
Context Diagram



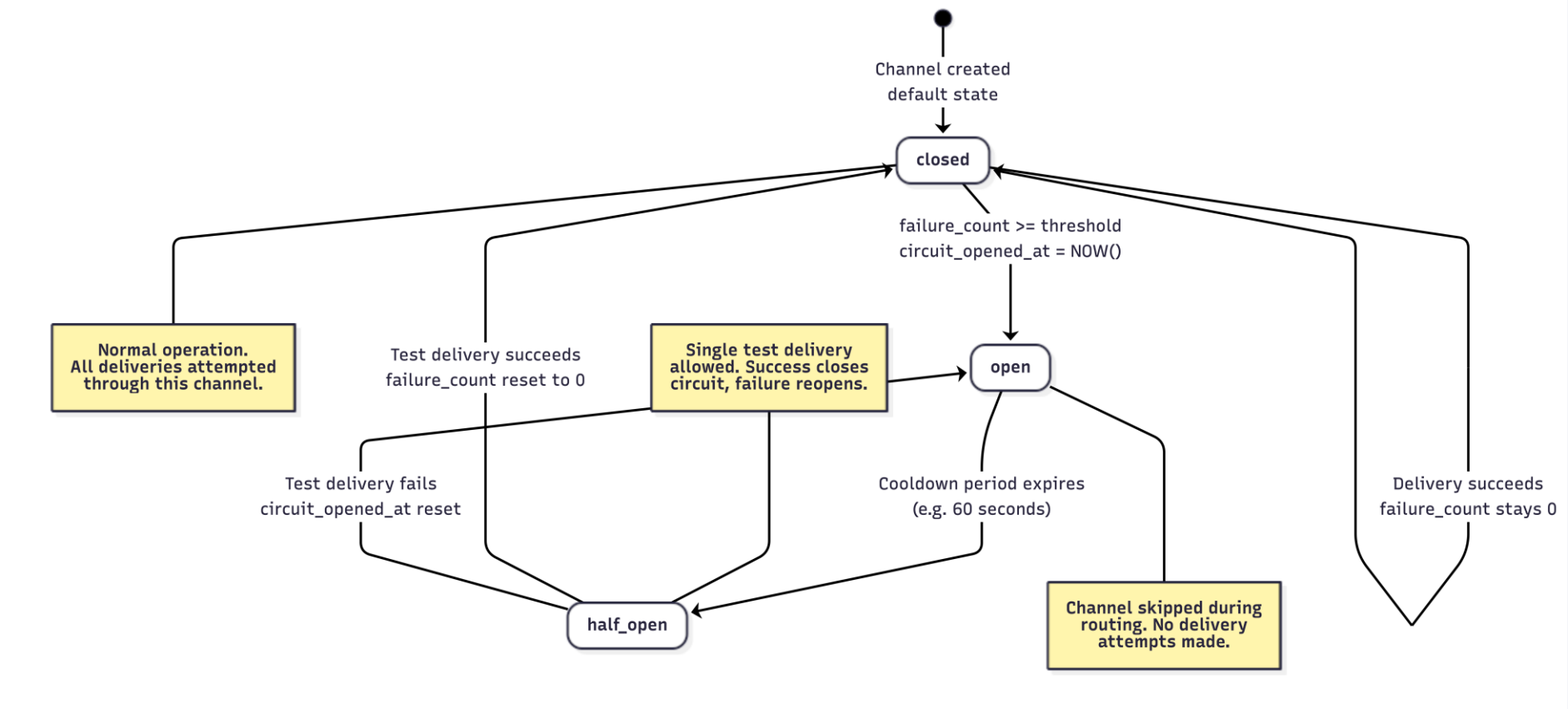
Sequence Diagram



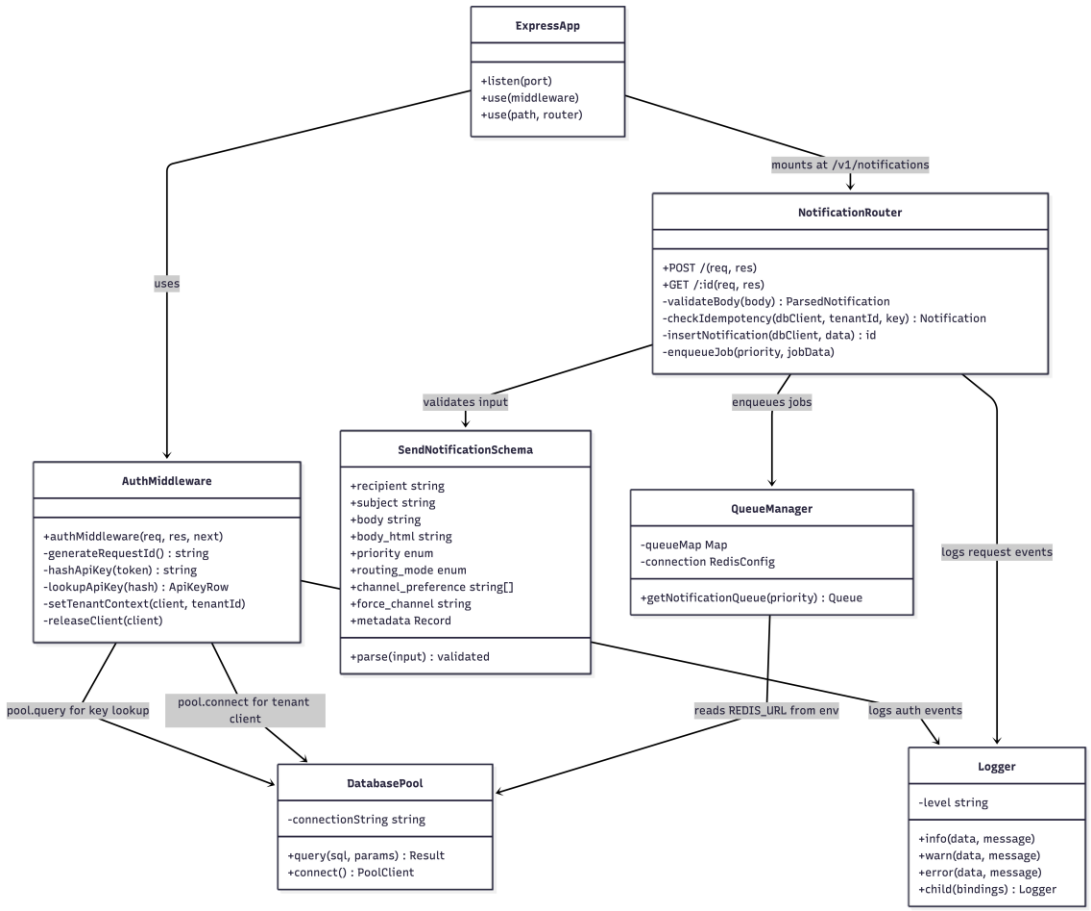
State Diagram - Notification Lifecycle



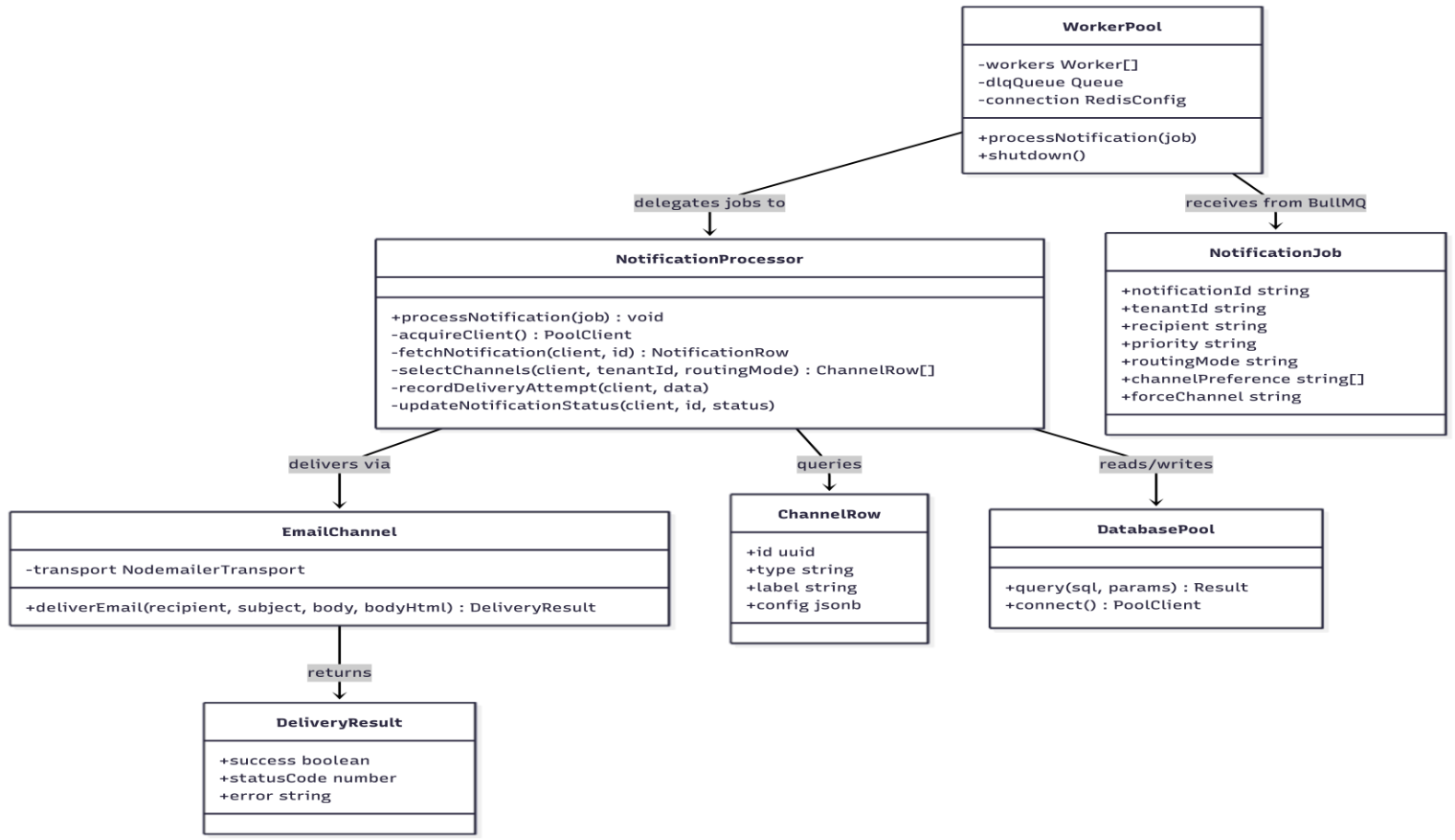
State Diagram - Circuit Breaker



Class Diagram - API Server



Class Diagram - Worker Service



Sprint 1 Recap

API Endpoints Live

POST /v1/notifications (202 async), GET /:id, GET /notifications (cursor pagination), GET /summary, POST /tenants/register

Email Delivery Working

Nodemailer integration with Mailpit for local dev. BullMQ job queue with retry and exponential backoff.

Dashboard Polling

React dashboard with stats cards and notifications table. TanStack Query polling every 30 seconds.

Integration Test Suite

14 acceptance verification steps covering auth, validation, async delivery, engagement tracking, and status queries. Delivery pipeline verified end-to-end.

Merged to Main

8 team PRs integrated with fixes. develop merged to main via PR #27. TruffleHog: 0 secrets detected.

Product Backlog

29 Story Points | 9 Stories

ID	Story	Pts	Ownership
PB-13	ML Service - XGBoost Training & Prediction API	8	Salvatore
PB-14	Socket.IO Dashboard Real-time Feed	5	Zeeya & Dhawalshree
PB-19	Adaptive Routing Integration (Worker + ML)	3	Salvatore
PB-15	Engagement Feedback Loop	3	Rhythm & Krish
PB-16	Fatigue Detection	3	Rhythm & Krish
PB-11	Rate Limiting (Redis)	3	Yusif
PB-12	CI/CD Activation (GitHub Actions)	2	Yusif
PB-20	Retro Action Item 1 - PR Self-Review Checklist	1	Team
PB-21	Retro Action Item 2 - Sprint Kickoff Walkthrough	1	Team

Sprint 2 Backlog - 29 Story Points Committed

ID	Pts	Story	Status
PB-13	8	ML Service - XGBoost Training & Prediction API	Done
PB-14	5	Socket.IO Dashboard Real-time Feed	Done
PB-19	3	Adaptive Routing Integration (Worker + ML)	Done
PB-15	3	Engagement Feedback Loop	Done
PB-16	3	Fatigue Detection	Done
PB-11	3	Rate Limiting (Redis)	Done
PB-12	2	CI/CD Activation (GitHub Actions)	Done
PB-20	1	Retro Action Item 1	Done
PB-21	1	Retro Action Item 2	Done

Sprint 2 Stories & Acceptance Criteria

ID	Feature	User Story	Acceptance Criteria	SP	Priority
PB-13	ML Service	As a developer, I want the ML model to predict the best channel per user so that engagement improves over time	• XGBoost returns probability scores per channel • Adaptive routing selects highest-probability (90%) • Exploration selects random (10%) • Different users get different routing	8	High
PB-14	Socket.IO Dashboard	As a developer, I want real-time delivery events in the dashboard so that I don't need to refresh	• Socket.IO persistent connection • New events appear instantly • Auto-reconnect on disconnect • Events: sent, delivered, failed	5	Medium
PB-19	Adaptive Routing	As a developer, I want the worker to call the ML service for channel selection so that routing adapts per recipient	• Worker calls ML service POST /predict • routing_decision column populated after delivery • Feature vector recorded per delivery attempt • Falls back to static if ML service unavailable	3	High
PB-15	Engagement Tracking	As a developer, I want email engagement tracked via tracking pixel so that the ML model has feedback data	• Tracking pixel embedded in email HTML • Pixel request logged as opened event • Engagement data stored in delivery_attempts • Feeds into ML training data	3	High
PB-16	Fatigue Detection	As a developer, I want the system to detect notification fatigue so that users are not over-notified	• Tracks notification count per user per 24h • Exceeding threshold flags user • Fatigue flag visible in API • Threshold configurable per tenant	3	Medium

Sprint 2 Stories & Acceptance Criteria

ID	Feature	User Story	Acceptance Criteria	SP	Priority
PB-11	Rate Limiting	As a developer, I want the API to enforce rate limits per tenant so that no single tenant can overload the system	• Rate limit tracked in Redis per tenant_id • Default: 1000 requests/minute • Exceeding returns 429 Too Many Requests • Remaining quota in response headers	3	High
PB-12	CI/CD Pipeline	As a developer, I want automated tests on every PR so that broken code cannot be merged	• GitHub Actions runs Jest + pytest on every PR • PR blocked if tests fail • TruffleHog scans for secrets • Test results visible in PR checks	2	Medium
PB-20	Retro Action 1	As a team, we want a PR self-review checklist so that code quality is verified before review	• PR template includes self-review checklist • Checklist covers tests, linting, security • PR cannot be submitted without checklist • Visible in GitHub PR description	1	Low
PB-21	Retro Action 2	As a team, we want a Sprint 2 kickoff code walkthrough so that all members understand the codebase	• Walkthrough session scheduled at sprint start • All team members attend • Key modules documented in CLAUDE.md • Recording or notes shared in Slack	1	Low

Test Cases - Sprint 2

✓ 12 / 12 Passed

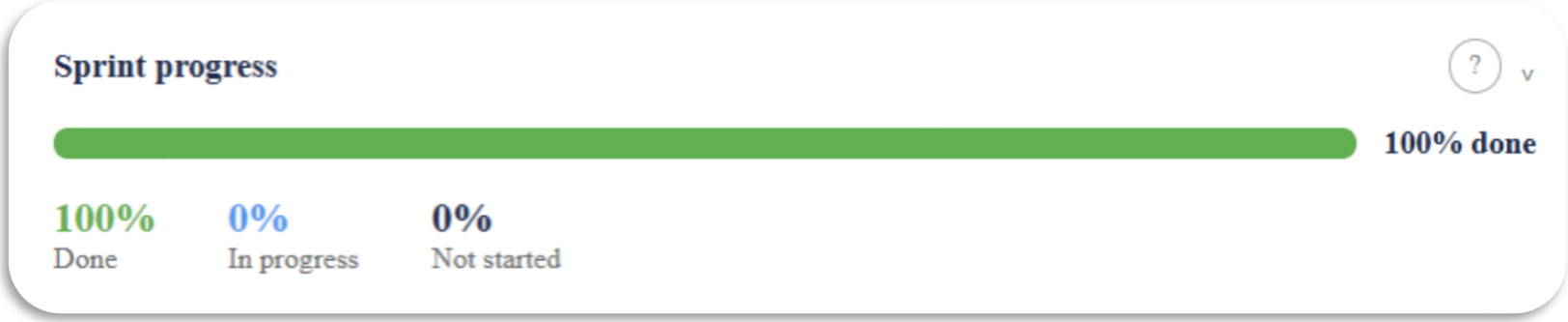
TC #	Test Case Title	Expected Input	Expected Output	Status
TC-01	**Tenant Registration Success**	Valid POST /v1/tenants/register payload	API key returned + tenant created	✓ Pass
TC-02	**API Key Hash Validation**	Tenant registration request	API key stored as SHA-256 hash	✓ Pass
TC-03	**Notification Creation Success**	Valid POST /v1/notifications payload	202 Accepted + notification ID	✓ Pass
TC-04	**Notification Validation Failure**	Missing required fields	400 with field-level validation errors	✓ Pass
TC-05	**Row-Level Security Enforcement**	Tenant A accessing Tenant B data	403 Forbidden	✓ Pass
TC-06	**BullMQ Job Enqueue**	Valid notification payload	Job appears in BullMQ queue	✓ Pass
TC-07	**Worker Processing SLA**	Enqueued BullMQ job	Job processed within 5s + status updated	✓ Pass
TC-08	**Channel Routing Logic**	Notification request	Channel selected by priority + logged reason	✓ Pass
TC-09	**Email Delivery Success**	Valid notification payload	Email received in Mailpit	✓ Pass
TC-10	**Delivery Attempt Logging**	Email delivery event	delivery_attempt stored with timestamps	✓ Pass
TC-11	**Get Notification Status**	GET /v1/notifications/:id	Full notification + delivery attempts returned	✓ Pass
TC-12	**Dashboard Real-Time Update**	Notification event sent	Dashboard receives live event via Socket.IO without refresh	✓ Pass

Sprint 2 - Stories Completed

SCRUM-188	GitHub Actions CI/CD Workflow	DevOps
SCRUM-182	Socket.IO Server Setup + Redis Pub/Sub Bridge	Backend
SCRUM-170	Adaptive Routing Worker Integration	Backend
SCRUM-171	Feature Vector Recording	Backend
SCRUM-172	routing_decision Population	Backend
SCRUM-164	XGBoost Training Pipeline	ML/Python
SCRUM-163	ML Service Prediction + Training API	ML/Python

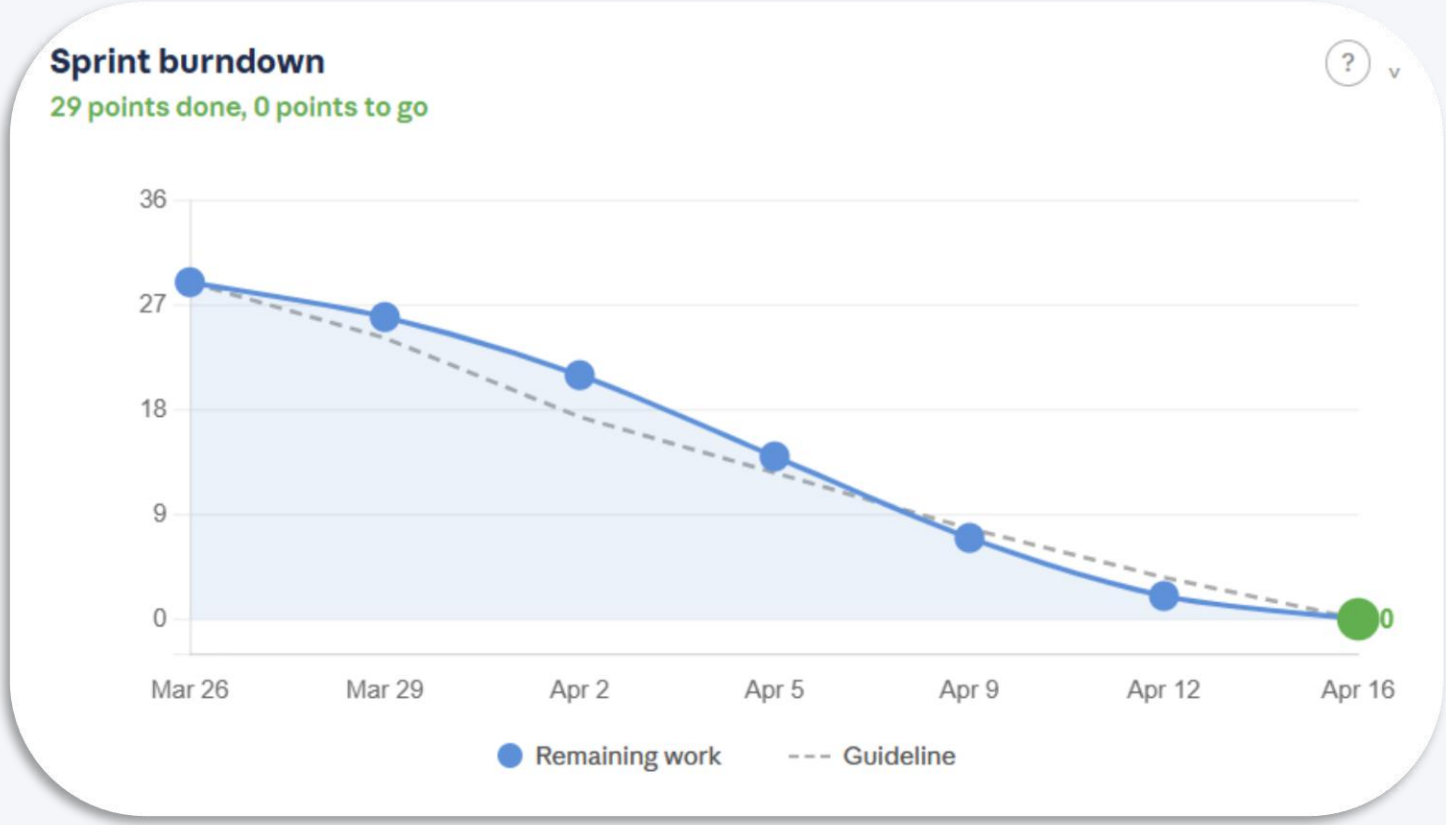
Sprint-2 Team Velocity

Total Estimated Points: 29
Total Points Delivered: 29
Total Points Carried Over: 0
Team Velocity: 29

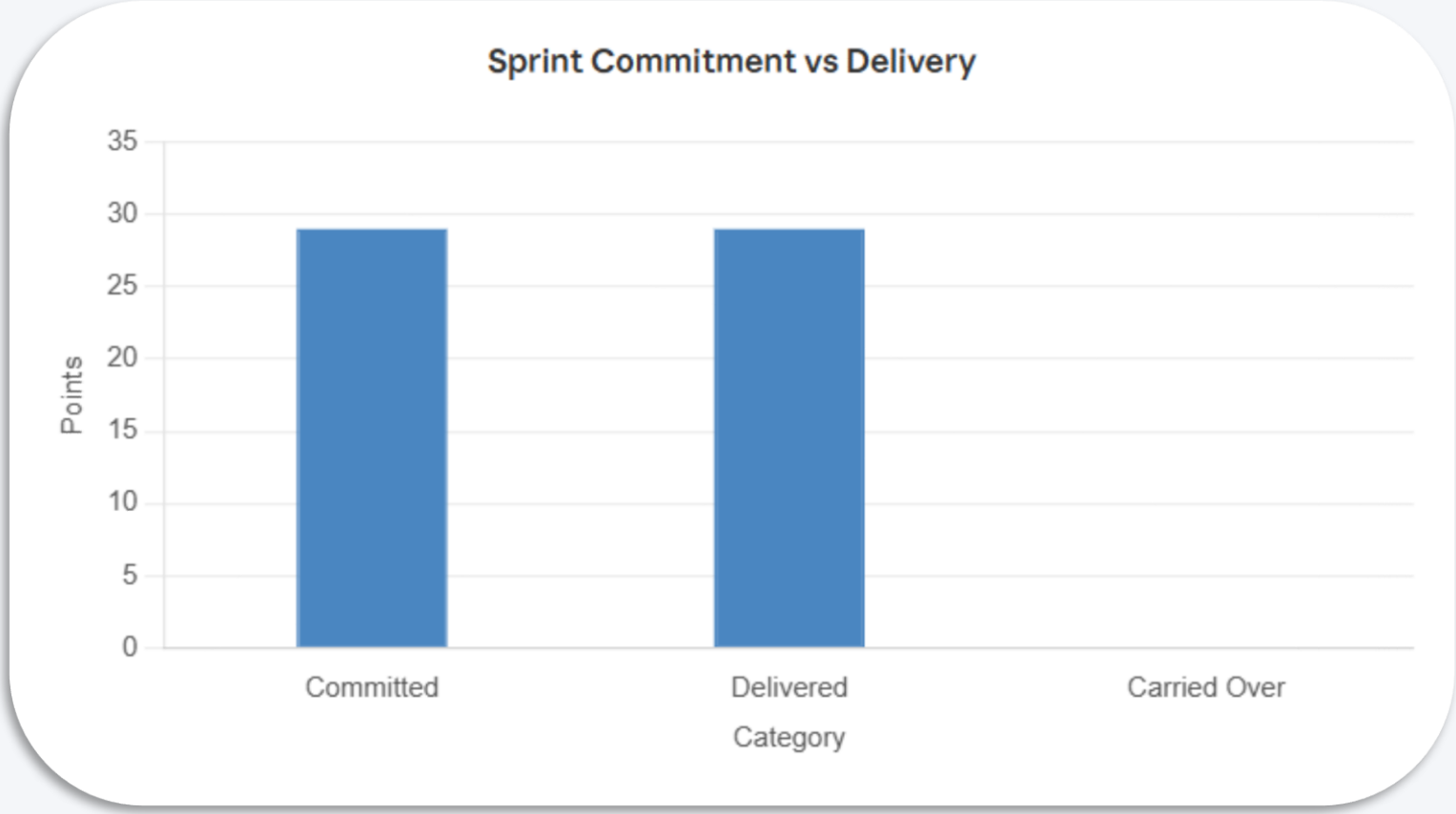


Historical & Average Velocity

Sprint-2 Burndown Chart



Completed/Committed Ratio



Sprint 2 Retrospective

What Went Well

- ML service integrated successfully
- CI/CD pipeline working
- Adaptive routing completed
- Socket.IO real-time feed working
- All 29 points delivered
- 12/12 tests passed
- Good team communication overall
- Team collaborated despite scheduling conflicts

What To Improve

- Scheduling conflicts reduced team meetups
- Socket.IO vs polling responsibility unclear
- Test coverage could be broader
- Sprint planning estimation needs refinement
- Documentation gaps across modules

Action Items

- Improved PR turnaround time
- Add more test cases for ML routing
- Better daily standups & sync schedule
- Document all modules before Sprint 3
- Clarify Socket.IO vs pulling ownership

Sprint 3 - Planned Scope

AWS Deployment

ECS Fargate, RDS, ElastiCache, S3 + CloudFront, ECR. Production-grade infrastructure.

Observability Stack

Prometheus metrics endpoint, Grafana dashboards, Loki log aggregation. All as Docker Compose services.

Interactive Dashboard

Send test notification form. Routing decision detail view. Full demo loop without terminal.

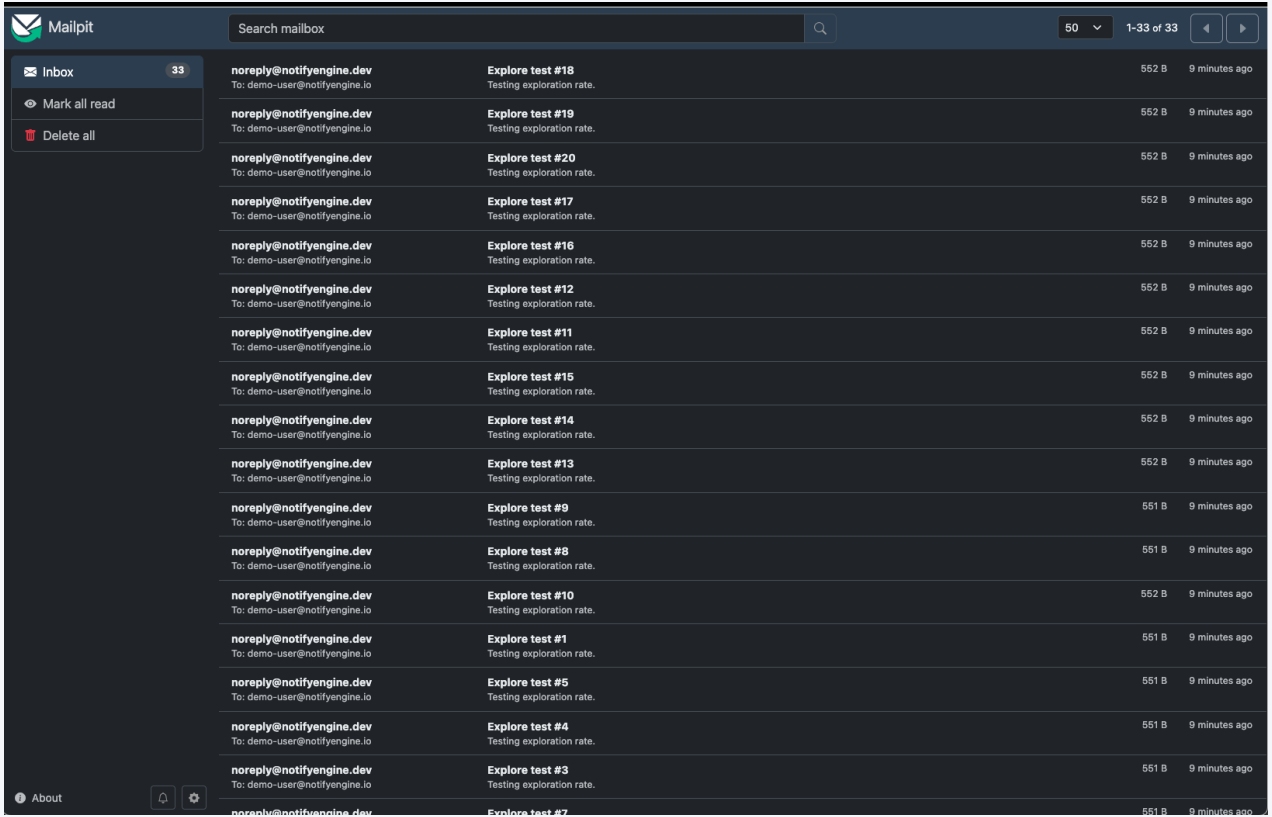
Circuit Breaker

Automatic channel fallback on provider failure. Health tracking per channel with half-open recovery.

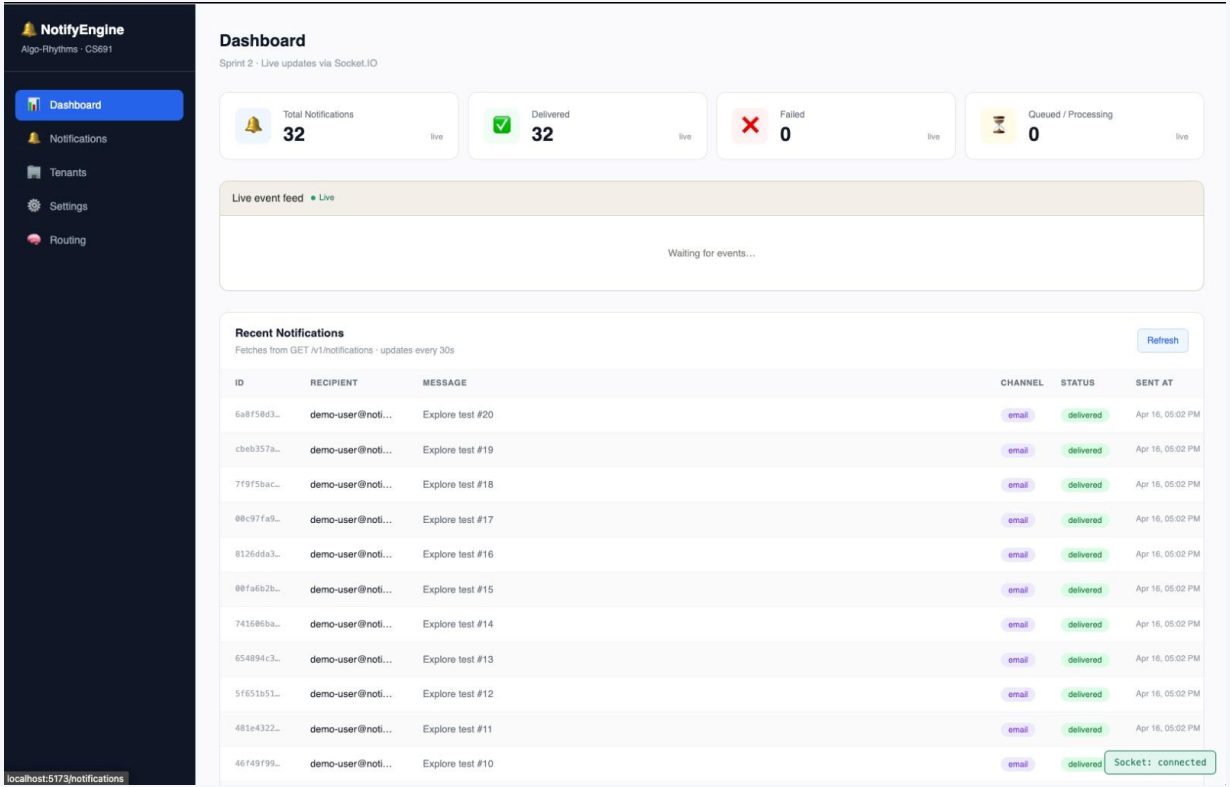
Documentation

Final technical paper. Deployment manual. User manual.

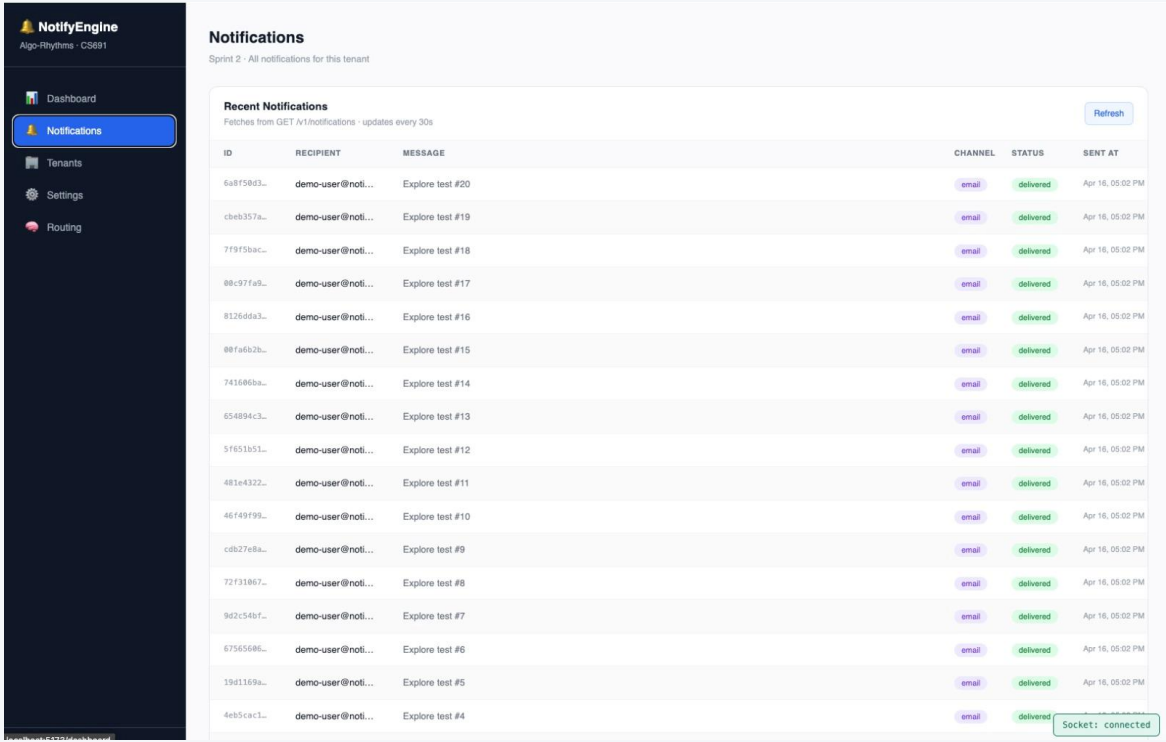
Demo - Application Screenshots



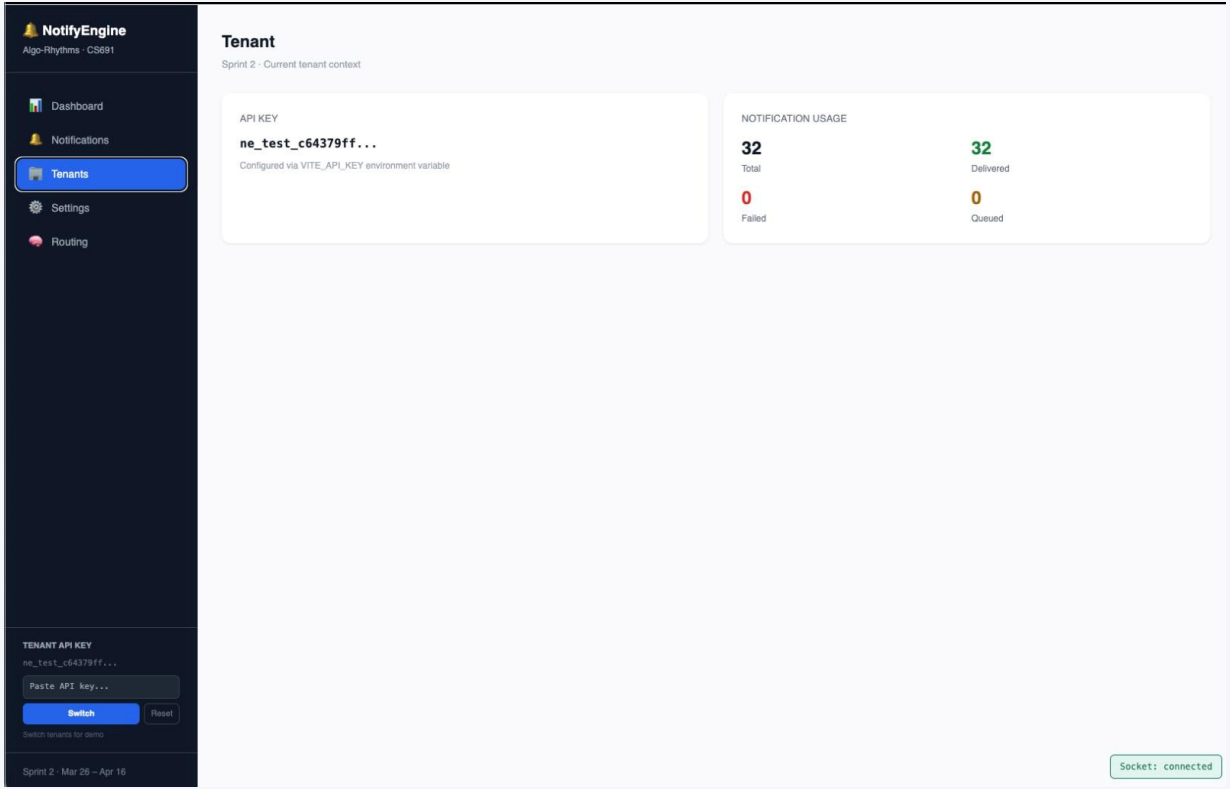
Demo - Application Screenshots



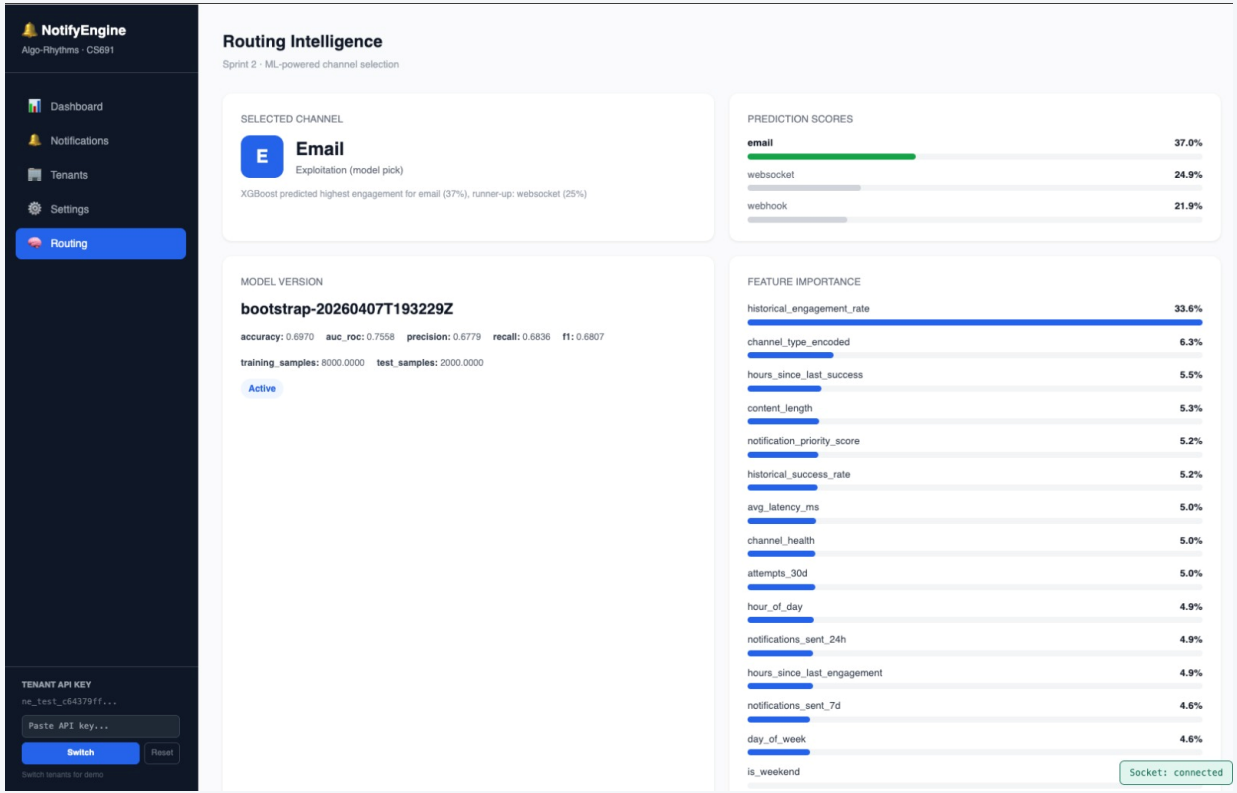
Demo - Application Screenshots



Demo - Application Screenshots



Demo - Application Screenshots



Demo - API Screenshots

```
PHASE 2 — FIRST NOTIFICATION

Sending the first notification with adaptive ML routing.
  The ML model will predict the best channel for demo-user@notifyengine.io.
  Since this is a new recipient with no history, predictions are
  based entirely on the bootstrap model trained on synthetic data.

  Watch the dashboard update in real-time at http://localhost:5173

  Press Enter to continue...

  ✓ Notification sent: 6433a0b1-5076-46c0-a366-60e8978f3f9b

Waiting for delivery...
  ✓ Delivered via email

Routing Decision (what the ML model recommended):
{
  "mode": "adaptive",
  "reason": "Exploration mode (10% chance): randomly selected email. Model would have chosen websocket (19%)",
  "selected": "email",
  "exploration": true,
  "predictions": {
    "email": 0.1303,
    "webhook": 0.1646,
    "websocket": 0.1891
  },
  "model_version": "bootstrap-20260407T193229Z"
}

Delivery Attempt (what actually happened):
  Attempt #1: channel=email  status=success  duration=197ms

Feature Vector (the 15 features the model used):
{
  "is_weekend": 0,
  "day_of_week": 3,
  "hour_of_day": 21,
  "attempts_30d": 0,
  "channel_type": "email",
  "avg_latency_ms": 1000,
  "channel_health": 1,
  "content_length": 49,
  "notifications_sent_7d": 0,
  "notifications_sent_24h": 0,
  "historical_success_rate": 0,
  "hours_since_last_success": 720,
  "historical_engagement_rate": 0,
  "hours_since_last_engagement": 720,
  "notification_priority_score": 2
}

Recipient Channel Stats (after 1 delivery):
  email: attempts=1  successes=1  engagements=0  avg_latency=65ms  sent_24h=1  sent_7d=1
  email: attempts=1  successes=1  engagements=0  avg_latency=55ms  sent_24h=1  sent_7d=1
  email: attempts=32  successes=32  engagements=4  avg_latency=26.53125ms  sent_24h=32  sent_7d=32
  email: attempts=32  successes=32  engagements=4  avg_latency=22.218749999999996ms  sent_24h=32  sent_7d=32
  email: attempts=1  successes=1  engagements=0  avg_latency=59ms  sent_24h=1  sent_7d=1
  email: attempts=32  successes=32  engagements=4  avg_latency=24.687500000000007ms  sent_24h=32  sent_7d=32
  email: attempts=1  successes=1  engagements=0  avg_latency=8ms  sent_24h=1  sent_7d=1
  email: attempts=32  successes=32  engagements=4  avg_latency=31.093749999999996ms  sent_24h=32  sent_7d=32
  email: attempts=32  successes=32  engagements=4  avg_latency=38.125ms  sent_24h=32  sent_7d=32
  email: attempts=1  successes=1  engagements=0  avg_latency=8ms  sent_24h=1  sent_7d=1
  email: attempts=32  successes=32  engagements=4  avg_latency=35.78125ms  sent_24h=32  sent_7d=32
  email: attempts=1  successes=1  engagements=0  avg_latency=60ms  sent_24h=1  sent_7d=1
  email: attempts=1  successes=1  engagements=0  avg_latency=197ms  sent_24h=1  sent_7d=1
  email: attempts=32  successes=32  engagements=4  avg_latency=48.59375ms  sent_24h=32  sent_7d=32
  email: attempts=1  successes=1  engagements=0  avg_latency=53ms  sent_24h=1  sent_7d=1
  Training samples in database: 366 total, 66 engaged
```

```
PHASE 7 — EXPLORATION (EPSILON-GREEDY)

Sending 20 rapid notifications to trigger the 10% exploration rate.
  The epsilon-greedy strategy means ~90% of routing decisions
  exploit the model's best prediction, while ~10% explore a
  random channel to discover changing user preferences.

  Press Enter to continue...

  Sent 20/20...

Waiting for delivery...

Exploration results:

  Exploited (model pick):  18/20
  Explored (random):       2/20

  ✓ Exploration rate is within expected range for epsilon=0.10

Example exploration reason:
  Exploration mode (10% chance): randomly selected websocket. Model would have chosen email (29%)

PHASE 8 — SUMMARY

  NOTIFYENGINE SPRINT 2 DEMO

  Total notifications:                    32
  Delivered:                              32
  Engaged (email opens):                   4
  Model version:              bootstrap-20260407
  Exploration rate:                 ~2/20 (10%)
  Tenant isolation:                   Confirmed
```

Live Application Demo

Docker Compose local stack demonstration-Sprint 2

SETUP

1. Start all services with docker-compose up -d
2. Run API, Worker, Dashboard with npm run dev

API CALL

3. Send POST /v1/notifications via PowerShell with Bearer API key
4. Watch worker terminal-ML service called, XGBoost selects channel, notification delivered

REAL-TIME

5. Dashboard live event feed updates via Socket.IO

EMAIL

6. Check Mailpit at localhost:8025 for delivered email

VERIFY

7. View routing_decision in GET /v1/notifications/:id response

Submitted recording to

YouTube GitHub

GitHub Repository

Link for GitHub

Turborepo monorepo with 4 services: API, Worker, ML Service, Dashboard

Branch strategy: feature/* and bugfix/* PRs against develop

Squash merges to develop, PR merges to main at sprint milestones

GitHub Actions CI: unit + integration + security scan on every PR

Infrastructure smoke test validates Docker Compose on develop/main merges

AI Disclosure

Tools Used

Claude (Anthropic) for architecture design, code generation, and documentation. Claude Code for local implementation with .claude-docs/ spec files as context.

Guardrails

CLAUDE.md (500+ lines) with security rules, coding conventions, verification rules, and 5 mandatory human validation checkpoints. Pre-commit hooks scan for secrets. TruffleHog runs on every PR via GitHub Actions.

How AI Is Applied

Architecture decisions documented with rationale before code generation. All generated code reviewed and tested. AI never pushes to remote - all git operations handled by humans.

What AI Does NOT Do

AI does not fabricate metrics, test results, or project status. AI does not make unilateral decisions. AI does not replace understanding - team members explain code they submit.

Thank You

Team Algo-Rhythms

CS 691 | Pace University | Spring 2026

NotifyEngine — ML-Powered Adaptive Notification Routing

Questions & Answers